terminal bench规则文档

数据集介绍

与传统的代码生成数据集（如 HumanEval）或静态问答数据集不同，Terminal-Bench 强调动态交互。模型不能仅仅输出一段代码，而是需要作为一个 Agent，通过不断向终端发送 Bash/Zsh 命令，观察终端的输出反馈，并根据反馈调整后续操作，直到完成最终任务。

任务地址
https://github.com/harbor-framework/terminal-bench-2

均考虑2.0的版本不考虑1.0的版本

数据集格式介绍

每个任务必须是一个独立目录：

代码块

```
<task_slug>/
├── task.toml
├── instruction.md
├── environment/
│   ├── Dockerfile
│   └── app/
├── solution/
│   └── solve.sh
└── tests/
    ├── test.sh
    └── test_outputs.py
```

- `task_slug` 使用小写字母、数字和短横线。
- `task.toml` 使用 Terminal-Bench 2.0 schema，`schema_version = "1.1"`。
- `instruction.md` 用英文写清楚输入、输出、格式、排序、错误处理和边界条件。
- `environment/Dockerfile` 的 build context 是 `environment/`。
- `environment/Dockerfile` 不允许 `COPY ../...`。
- `solution/solve.sh` 是 oracle 参考解法，只能修复 `/app` 中的项目文件。
- `tests/test.sh` 必须写 `/logs/verifier/reward.txt`。

- `tests/test_outputs.py` 必须验证行为，不能读取 `solution/solve.sh`、`/app/solve.sh` 或 `solution/` 目录。
- 任务必须可离线运行，不能依赖外部网络、私有 API、登录态或随机结果。

运行指南

进入任务目录

执行docker build

代码块

```
docker build environment/Dockerfile  -t tag
```

这里的tag要和task.toml中[environment]的docker_image的值一致

例如:

代码块

```
[environment]
build_timeout_sec = 600.0
docker_image = "generated-terminal-bench-hard-ledger-reconciliation-engine:20260610"
cpus = 1
memory_mb = 2048
storage_mb = 10240
gpus = 0
allow_internet = true
mcp_servers = []
```

代码块

```
docker build environment/Dockerfile  -t generated-terminal-bench-hard-ledger-reconciliation-engine:20260610
```

python环境下安装harbor(uv、conda等)

代码块

```
uv run harbor trial start -p . -a oracle
```

此命令会验证 `solution/solve.sh` 执行以后所有的测试是否通过，也就是 `/logs/verifier/reward.txt` 是否是1。

代码块

```
uv run harbor trial start `
  -p . `
  -a claude-code `
  -m model `
  --ae ANTHROPIC_API_KEY=你的Key `
  --ae ANTHROPIC_AUTH_TOKEN=你的Key `
  --ae ANTHROPIC_BASE_URL=https://你的BaseUrl `
  --ae ANTHROPIC_MODEL=model
```

此命令会调用模型进行修复最后在jobs文件夹下查看是否 `/logs/verifier/reward.txt` 是1。

注意事项

如果使用deepseek等输出为utf-8的模型

找到harbor源代码 harbor/agents/installed/claude_code.py

找到函数

代码块

```
def _convert_events_to_trajectory(self, session_dir: Path) -> Trajectory |
None:
```

修改

代码块

```
with open(session_file, "r", encoding="gbk", errors="replace") as handle:
```

为

代码块

```
with open(session_file, "r", encoding="utf-8", errors="replace") as handle:
```

可以查看报错的日志如果出现gbk报错即可用这个方法解决

验收标准：

1. 抽检：采购方随机抽取200条数据，供应商提供对应的运行环境和测试用例用于评估，每条数据字段符合要求，交付轨迹使用DeepSeek V4-pro-max或Claude-Code Opus4.6/4.7模型推理完整慢思考轨迹，难度评分根据数据源正确率统计模型对应采用DeepSeek V4-pro-max、Claude-Code Opus4.6/4.7、GLM-5、opus4.5、GPT-5.2、Gemini3-pro模型进行慢思考推理，4次回复准确率大于0小于等于50%（答对1-2次）即为通过。样本整体通过率达到98%，视为整批次通过；

2.整体检查：任务的领域、技能分布涵盖Terminal Bench2.0评测集所涉及领域和技能，整体字段格式要求通过率100%且任务查重通过比例不低于98%，视为整批次验收通过